UNIVERSITI TEKNOLOGI MARA
MARKING SCHEME

COURSE	:	COMPUTER ORGANIZATION AND ARCHITECTURE
COURSE CODE	:	CSC429
EXAMINATION	:	NOVEMBER 2026 (MOCK)
TIME	:	3 HOURS

MARKING SCHEME NOTES

1. This marking scheme accompanies the CSC429 mock paper (November 2026).
   Answers are unofficial reference answers, not official UTM marking schemes.

2. Marking points show the expected answer and mark allocation; check them against the source paper.

3. Part A = 4 x 5 + 6 x 10; Part B = 1 x 20; total = 100 marks.

4. Approved real questions only; no generators, placeholders or parametric variants.
   Questions and answers are for local study use only.

5. Structure derived from the supplied CSC429 papers; every question remains traceable to its source page.

FOR MARKERS' USE ONLY

This marking scheme consists of 11 printed pages

© Hak Cipta Universiti Teknologi MARA

PART A — MARKING GUIDE

QUESTION 1 (5 marks)

Source: CSC429-2023-07, page [2]

Unofficial model answer:

a) Convert both operands to binary, add, then convert the sum to base 2. 52.73_8 = 101 010 . 111 011_2 = 101010.111011_2. $AB.2_{16}$ = 1010 1011 . 0010_2 = 10101011.0010_2. Align and add: 0101010.111011 + 10101011.0010 = 11100000.000011_2 (check: 42.921875 + 171.125 = 214.046875 = 11010110.000011_2? Recompute: 52.73_8 = 5×8+2 + 7/8+3/64 = 42 + 0.921875 = 42.921875; $AB.2_{16}$ = 10×16+11 + 2/16 = 171 + 0.125 = 171.125; sum = 214.046875. 214 = 11010110_2, 0.046875 = 3/64 = 0.000011_2, so sum = 11010110.000011_2. The binary addition must be redone: 101010.111011 + 10101011.001000 = 11010110.000011_2.) Thus 52.73_8 + $AB.2_{16}$ = 11010110.000011_2. b) Convert 171.28125_{10} to binary: 171 = 10101011_2; 0.28125 = 0.01001_2 (0.28125×2=0.5625→0; ×2=1.125→1; ×2=0.25→0; ×2=0.5→0; ×2=1.0→1), so 171.28125_{10} = 10101011.01001_2. Add 10101100.11_2: align 10101011.01001 + 10101100.11000 = 101010111.10001_2? Compute decimal: 10101100.11_2 = 172.75; sum = 344.03125. 344 = 101011000_2, 0.03125 = 0.00001_2, so sum = 101011000.00001_2. Binary addition: 10101011.01001 + 10101100.11000 = 101010111.10001_2? That is 343.53125, so error. Correct: 10101011.01001 + 10101100.11000 = 101010111.10001_2? Let's add: fractional .01001 + .11000 = 1.00001, carry 1; integer 10101011 + 10101100 + 1 = 101010111? 171+172+1=344, binary 101011000. So result 101011000.00001_2. Therefore 171.28125_{10} + 10101100.11_2 = 101011000.00001_2 (base 8: 530.02_8).

Marking points:

- *1 mark: correct conversion of 52.73_8 to binary 101010.111011_2*
- *1 mark: correct conversion of $AB.2_{16}$ to binary 10101011.0010_2 and correct binary addition yielding 11010110.000011_2*
- *1 mark: correct conversion of 171.28125_{10} to binary 10101011.01001_2*
- *1 mark: correct binary addition of 10101011.01001_2 and 10101100.11_2 yielding 101011000.00001_2*
- *1 mark: complete workflow shown with alignment and carry/borrow details for both parts*

Model confidence: medium.

Model note: The question text shows '--.---------.-.--2' and '8.' as placeholders; assumed the required result bases are binary for (a) and binary (or octal) for (b). The final answers are 11010110.000011_2 for (a) and 101011000.00001_2 for (b). If a different target base was intended, convert accordingly. This is an unofficial model answer, not an official UTM marking scheme.

QUESTION 2 (5 marks)

Source: CSC429-2016-06, page [3]

Unofficial model answer:

a) AI was introduced in the fifth generation of computers. The fifth generation (from the 1980s onward) is characterised by parallel processing, very-large-scale integration (VLSI), and a focus on artificial intelligence and natural-language processing.

b) The programming language most associated with this technology is Prolog (PROgramming in LOGic), a logic programming language widely used for AI and expert systems. LISP is also acceptable as an early AI language, but Prolog is the standard answer for fifth-generation AI.

c) The advantage of embedding AI in everyday life is that it automates routine and complex tasks, improving convenience, efficiency and decision-making. For example, smart-home systems adjust lighting and temperature automatically, intelligent tutoring systems personalise learning, and robots perform dangerous or repetitive jobs, thereby saving time and reducing human error.

© Hak Cipta Universiti Teknologi MARA

d) One possible scenario where AI can harm people is an autonomous vehicle or robot malfunctioning due to a software error or flawed training data, causing a collision or physical injury. Another scenario is an AI-based medical diagnosis system giving an incorrect recommendation that leads to wrong treatment. The key point is that AI lacks human judgement and accountability, so errors can directly endanger safety or privacy.

Marking points:

• 1 mark: (a) Fifth generation stated.

• 1 mark: (b) Prolog (or LISP) named as the AI programming language.

• 1 mark: (c) Valid advantage of embedding AI (e.g., automation, convenience, efficiency, personalisation) with a brief explanation.

• 2 marks: (d) One plausible harmful scenario clearly described (1 mark for the scenario, 1 mark for explaining how it harms people).

Model confidence: high.

Model note: The question text contains a typo 'A1' for 'AI'. The answer assumes standard CSC429 course material where the fifth generation is associated with AI and Prolog is the representative language. No figure or table was supplied.

QUESTION 3 (5 marks)

Source: CSC429-2024-07, page [2]

Unofficial model answer:

a) Two reasons why binary is used instead of decimal in computer architecture: (1) Physical implementation with two-state devices: Digital circuits are most reliably built from components that have two stable states, such as a transistor that is either cut-off (off, logic 0) or saturated (on, logic 1), or a voltage level that is either low or high. Distinguishing ten distinct voltage levels (as decimal would require) would demand precise analogue thresholds, making circuits slower, more error-prone and more expensive. (2) Boolean algebra and logic simplification: Binary maps directly onto Boolean logic (true/false, 1/0), so combinational and sequential circuits can be designed, minimised (e.g. Karnaugh maps) and implemented using simple AND, OR and NOT gates. Arithmetic such as addition and subtraction also reduces to straightforward bitwise rules, giving simpler, faster and more reliable hardware than decimal arithmetic. b) Workflow to estimate vacuum tubes for 79 in binary: Step 1: Convert 79 to binary by repeated division by 2, recording remainders: 79/2 = 39 remainder 1 (LSB); 39/2 = 19 remainder 1; 19/2 = 9 remainder 1; 9/2 = 4 remainder 1; 4/2 = 2 remainder 0; 2/2 = 1 remainder 0; 1/2 = 0 remainder 1 (MSB). Reading remainders from bottom to top gives $79 = 1001111_2$. Step 2: Count the bits: 1001111 has 7 bits. Step 3: In a first-generation vacuum-tube computer, one vacuum tube (typically a flip-flop or equivalent bistable circuit) is needed to store one bit. Therefore the number of vacuum tubes needed is 7. (If a fixed 8-bit byte were assumed, 8 tubes would be required; the minimal representation needs 7.)

Marking points:

• 1 mark: reason 1 – two-state devices (on/off, high/low) are simple and reliable to build in hardware

• 1 mark: reason 2 – binary maps directly to Boolean logic / simplifies circuit design and arithmetic

• 1 mark: correct repeated-division workflow shown for converting 79 to binary

• 1 mark: correct binary result 1001111_2

• 1 mark: correct count of 7 bits and conclusion that 7 vacuum tubes are needed (accept 8 if fixed 8-bit word explicitly justified)

Model confidence: high.

Model note: Assumes one vacuum tube stores one bit (flip-flop equivalent), as is standard for first-generation computer examples. If the course defines a fixed word length (e.g. 8 bits), the answer may be 8 tubes; the minimal representation is 7. No figure or table was supplied.

© Hak Cipta Universiti Teknologi MARA

QUESTION 4 (5 marks)

Source: CSC429-2018-06, page [5]

Unofficial model answer:

a) One characteristic of an SMP is that all processors share a single common main memory and a single copy of the operating system, with uniform memory access time from any processor to any memory location. (Equivalently: two or more identical processors connected to a shared memory and I/O subsystem.)

b) 'Symmetric' means that all processors are treated as peers: they are identical in capability, have equal access to shared memory and I/O, and any processor can execute any task, including kernel/OS code. There is no master-slave relationship; the OS can schedule any process on any processor, and each processor has the same view of memory and devices.

c) One advantage of SMP organization is scalability/throughput: additional processors can be added to increase performance (e.g., more concurrent processes/threads) while the system remains a single coherent machine with one OS image, and it can provide graceful degradation if one processor fails (depending on OS support).

Marking points:

• 1 mark (a): Correctly states one SMP characteristic, e.g., shared single main memory and single OS image, or identical processors with uniform memory access.

• 2 marks (b): Explains 'symmetric' as all processors being peers/identical with equal access to shared memory and I/O and no master-slave hierarchy (1 mark for peer/equal treatment; 1 mark for equal access to shared resources/any processor can run any task including OS).

• 1 mark (c): Gives one valid advantage, e.g., scalability/increased throughput, or graceful degradation/fault tolerance.

• 1 mark: Overall clarity and correct use of terminology (e.g., shared memory, uniform memory access, peer processors).

Model confidence: high.

Model note: Unofficial model answer; not an official UTM marking scheme. No figure/table was supplied. Mark allocation is an interpretation of the 5-mark total.

QUESTION 5 (10 marks)

Source: CSC429-2016-12, page [5]

Unofficial model answer:

a) The input/output (I/O) problem arises because the CPU, memory, and peripheral devices differ greatly in speed, data format, and control method, so they cannot simply be connected and run in lockstep. Peripherals are typically much slower than the CPU (e.g., a keyboard or printer versus a multi-GHz processor), have different data widths and signal levels, and may be electromechanical or asynchronous. Without an I/O module to act as an intermediary, the CPU would have to wait for slow devices, wasting cycles, and would need detailed device-specific knowledge. The I/O module resolves this by providing a standard interface to the CPU/system bus, buffering data to absorb speed differences, performing format conversion (e.g., serial-to-parallel), and handling device control and status, thereby freeing the CPU from low-level device management.

b) The three categories of I/O modules are: (1) Programmed I/O module: the CPU directly controls the transfer and polls device status; the CPU issues a read/write command and waits, so it is simple but inefficient. Example: a simple keyboard controller or a basic parallel port where the CPU repeatedly checks a status register before reading a character. (2) Interrupt-driven I/O module: the module signals the CPU with an interrupt when a device is ready or a transfer completes, so the CPU need not poll; the CPU executes an interrupt service routine to transfer data. Example: a network interface card (NIC) that raises an interrupt when a packet arrives, or a UART that interrupts on received data. (3) Direct Memory Access (DMA) module: the module transfers blocks of data directly between the device and main memory without CPU involvement per

© Hak Cipta Universiti Teknologi MARA

word, issuing a single interrupt at completion; this suits high-speed block devices. Example: a disk controller performing DMA to load a file into memory, or a graphics card DMA-ing frame data. (Some texts classify by data transfer mode as programmed, interrupt-driven, and DMA; others classify by module function as I/O controller, DMA controller, and channel/I/O processor. The above uses the common transfer-mode classification.)

Marking points:

• 1 mark: I/O problem occurs because of speed mismatch between CPU/memory and peripherals.

• 1 mark: I/O problem occurs because of differences in data format, signal levels, or control methods requiring conversion and device-specific handling.

• 1 mark: I/O module acts as intermediary/interface to resolve these differences (buffering, format conversion, control/status).

• 1 mark: Correctly names first category (e.g., programmed I/O).

• 1 mark: Correct description of first category (CPU polls/controls, inefficient).

• 1 mark: Appropriate example for first category (e.g., keyboard/parallel port polling).

• 1 mark: Correctly names second category (e.g., interrupt-driven I/O).

• 1 mark: Correct description of second category (device interrupts CPU when ready).

• 1 mark: Appropriate example for second category (e.g., NIC/UART interrupt).

• 1 mark: Correctly names third category (e.g., DMA).

• 1 mark: Correct description of third category (block transfer direct to memory, CPU bypassed).

• 1 mark: Appropriate example for third category (e.g., disk controller DMA).

Model confidence: high.

Model note: The question does not specify a classification scheme; the answer uses the common programmed/interrupt-driven/DMA transfer-mode categories. If the lecturer's scheme is functional (I/O controller, DMA controller, channel/I/O processor), the same structure applies with different examples. No figure or table was supplied.

QUESTION 6 (10 marks)

Source: CSC429-2018-01, page [4, 5]

Unofficial model answer:

a) Ranking by price (cheapest to most expensive per bit): tape back-up, hard drive, cache, register. Tape is the cheapest per bit (slowest, offline/sequential backup medium), followed by the hard drive (magnetic disk, cheaper per bit than semiconductor memory), then cache (fast SRAM, expensive per bit), and finally registers (fastest, most expensive per bit, built into the CPU). This ordering follows the memory hierarchy: as speed increases, cost per bit increases and capacity decreases.

b) One benefit of cache memory: it reduces the average memory access time (and hence improves overall system performance) by exploiting locality of reference. Because cache is small, fast SRAM located close to the CPU, frequently used instructions and data (temporal and spatial locality) are kept there, so most accesses are satisfied from cache rather than the slower main memory, reducing the effective memory latency and the number of main-memory accesses.

c) Write through vs write back: In write through, every write by the CPU is written to both the cache block and the corresponding main-memory location simultaneously, so main memory is always kept consistent (up to date) with the cache; this is simple but generates more main-memory traffic. In write back, the write is made only to the cache block, and the main-memory copy is updated only when that dirty block is later replaced/evicted from the cache (a dirty bit marks modified blocks); this reduces main-memory write traffic but requires a dirty bit and makes main memory temporarily inconsistent with cache, and coherence must be managed.

Marking points:

• a) 4 marks: 1 mark each for correct position of tape back-up (cheapest), hard drive, cache, register (most

expensive); full 4 marks only if the whole order is correct.

• b) 2 marks: 1 mark for stating the benefit (e.g. reduces average memory access time / improves performance / exploits locality), 1 mark for a correct explanation of how it is achieved.

• c) 4 marks: 2 marks for correct description of write through (write to both cache and main memory, memory always consistent, more traffic), 2 marks for correct description of write back (write only to cache, update memory on eviction using dirty bit, less traffic but possible inconsistency).

• Total: 10 marks.

Model confidence: high.

Model note: Unofficial model answer, not an official UTM marking scheme. Part (a) assumes price per bit as the standard interpretation of 'price' in the memory hierarchy; if the question intended total cost of the device, the ranking could differ, but the hierarchy-based per-bit interpretation is standard for CSC429.

QUESTION 7 (10 marks)

Source: CSC429-2016-06, page [3]

Unofficial model answer:

a) To convert -197.0078125$_{10}$ to IEEE 754 single precision: sign bit = 1 (negative). Magnitude 197.0078125 = 197 + 0.0078125. 197$_{10}$ = 11000101$_2$. 0.0078125 = 1/128 = 2^{-7}, so binary fraction = 0.0000001$_2$. Thus 197.0078125$_{10}$ = 11000101.0000001$_2$. Normalize: 1.100010100000001$_2$ × 2^7. Exponent field = 7 + 127 = 134 = 10000110$_2$. Mantissa (fraction after leading 1) = 100010100000001 followed by zeros to 23 bits: 10001010000000100000000. Combine: sign 1, exponent 10000110, mantissa 10001010000000100000000 → binary 1 10000110 10001010000000100000000. Group into hex nibbles: 1100 0011 0100 0101 0000 0001 0000 0000 = C3450100$_{16}$. Answer: C3450100h.

b) 440AE800h is a 32-bit IEEE single precision pattern. Convert to binary: 0100 0100 0000 1010 1110 1000 0000 0000. Sign bit = 0 (positive). Exponent field = 10001000$_2$ = 136$_{10}$. Unbiased exponent = 136 − 127 = 9. Mantissa fraction = 00010101110100000000000$_2$. Significand = 1.000101011101$_2$. Value = 1.000101011101$_2$ × 2^9. Binary point shift 9 places: 1000101011.101$_2$. Convert integer part: 1000101011$_2$ = 512 + 32 + 8 + 2 + 1 = 555. Fractional part: 0.101$_2$ = 1/2 + 1/8 = 0.625. Total = 555.625$_{10}$. Answer: +555.625.

Marking points:

• 1 mark: sign bit = 1 for negative number

• 1 mark: correct binary representation of integer part 197 = 11000101

• 1 mark: correct binary representation of fractional part 0.0078125 = 0.0000001

• 1 mark: normalized form 1.100010100000001 × 2^7

• 1 mark: exponent field = 134 = 10000110

• 1 mark: correct 23-bit mantissa 10001010000000100000000

• 1 mark: final hexadecimal C3450100h

• 1 mark: part b sign positive and exponent field 136 → unbiased 9

• 1 mark: significand 1.000101011101 and binary value 1000101011.101

• 1 mark: decimal result 555.625

Model confidence: high.

Model note: The question text uses subscript 10 for decimal and h for hexadecimal; interpreted accordingly. No figure or table supplied. This is an unofficial model answer, not an official UTM marking scheme.

QUESTION 8 (10 marks)

Source: CSC429-2015-12, page [5]

Unofficial model answer:

a) Device labelling with appropriate I/O module: i. Satellite dish — input module (receives signals from space; can also be considered a communication input device). ii. Thumb drive — input/output module (storage device;

can be read from and written to). iii. Touch screen — input/output module (accepts touch input and displays output). iv. Joy stick — input module (provides positional input). v. Central processing unit (CPU) — not an I/O module; it is the processor that controls I/O via the I/O subsystem. vi. Modem — input/output module (modulator-demodulator; converts between digital and analog signals for bidirectional communication).

b) DMA (Direct Memory Access) is a technique where a dedicated DMA controller transfers data directly between an I/O device and main memory without continuous CPU involvement. The CPU initializes the DMA controller with the starting memory address, transfer count, and direction, then the DMA controller manages the transfer and interrupts the CPU upon completion. This improves overall computer performance by offloading data movement from the CPU, allowing the CPU to execute other instructions concurrently, reducing CPU idle time during I/O, and enabling high-speed block transfers that are more efficient than programmed I/O or interrupt-driven I/O for large data volumes.

c) (No subpart c text provided; answer omitted.)

Marking points:

- *1 mark: Satellite dish correctly labelled as input module*
- *1 mark: Thumb drive correctly labelled as input/output module*
- *1 mark: Touch screen correctly labelled as input/output module*
- *1 mark: Joy stick correctly labelled as input module*
- *1 mark: CPU correctly identified as not an I/O module (processor)*
- *1 mark: Modem correctly labelled as input/output module*
- *1 mark: DMA defined as direct data transfer between I/O and memory without CPU involvement*
- *1 mark: DMA controller initialized by CPU with address/count/direction*
- *1 mark: CPU freed to perform other tasks during DMA transfer*
- *1 mark: Overall performance improvement explained (e.g., reduced CPU overhead, faster block transfers)*

Model confidence: medium.

Model note: Subpart c is missing from the question text; only parts a and b are answered. The labelling of devices as input/output may vary slightly by convention (e.g., satellite dish could be considered a communication device), but the provided answers follow standard I/O module classification. This is an unofficial model answer, not an official UTM marking scheme.

QUESTION 9 (10 marks)

Source: CSC429-2019-12, page [4]

Unofficial model answer:

Registers are small, high-speed storage locations inside the CPU used to hold data, instructions, and addresses during program execution. Their specific functions are: (1) Program Counter (PC): holds the address of the next instruction to be fetched; it is automatically incremented after each fetch so the CPU knows where to continue. (2) Instruction Register (IR): holds the instruction currently being decoded and executed; its contents are fed to the control unit. (3) Memory Address Register (MAR): holds the address of the memory location to be accessed (read or written) by the memory unit. (4) Memory Data Register (MDR) / Memory Buffer Register (MBR): holds the data read from or to be written to memory; it acts as a buffer between the CPU and main memory. (5) Accumulator (AC): a general-purpose register that holds intermediate arithmetic and logic results and often one operand of an ALU operation. (6) General-Purpose Registers (GPRs): a set of registers (e.g., R0–R15) used by the programmer for holding operands, intermediate values, and addresses; they reduce memory accesses. (7) Status/Flag Register (PSW): holds condition flags (zero, carry, sign, overflow) and control bits that record the outcome of ALU operations and CPU state. (8) Stack Pointer (SP): holds the address of the top of the stack in memory, used for subroutine calls, returns, and push/pop operations. (9) Index Register: holds an offset used in indexed addressing to compute effective addresses. (10) Temporary registers (e.g., in the datapath) hold operands and results during instruction execution inside the CPU. Each register is typically a fixed-width flip-flop array directly accessible by the control unit, giving fast access compared with cache or main memory.

© Hak Cipta Universiti Teknologi MARA

Marking points:

• 1 mark: Correct definition of a register as small, fast CPU storage.

• 1 mark: Program Counter (PC) – holds address of next instruction.

• 1 mark: Instruction Register (IR) – holds current instruction being decoded/executed.

• 1 mark: Memory Address Register (MAR) – holds memory address for access.

• 1 mark: Memory Data Register (MDR/MBR) – buffers data to/from memory.

• 1 mark: Accumulator (AC) – holds intermediate ALU results/operand.

• 1 mark: General-purpose registers – programmer-visible operands/intermediate values.

• 1 mark: Status/Flag register (PSW) – condition flags and CPU state.

• 1 mark: Stack Pointer (SP) – points to top of stack for calls/returns.

• 1 mark: Any other valid register (index, temporary, instruction pointer) with correct function.

Model confidence: high.

Model note: No figure or table was supplied; answer is based on standard computer-organization register terminology. Marking assumes one mark per correctly described register/function up to 10 marks; if the official scheme groups registers differently, adjust allocation accordingly. This is an unofficial model answer, not an official UTM marking scheme.

QUESTION 10 (10 marks)

Source: CSC429-2018-12, page [2]

Unofficial model answer:

a) ENIAC belongs to the first generation of computers (roughly 1940–1956), which was characterised by vacuum-tube technology, machine-language programming and very large physical size.

b) ENIAC was built by John Presper Eckert and John William Mauchly at the Moore School of Electrical Engineering, University of Pennsylvania (with U.S. Army funding; the statement's 'Mr. X and Mr. Y' refers to these two designers).

c) The main technology used in ENIAC was the vacuum tube (thermionic valve); it used about 18,000 vacuum tubes together with relays, resistors and capacitors for its logic and storage.

d) Possible solutions to the problems faced when using ENIAC: Produced a lot of heat – use forced-air cooling/fans, air-conditioned machine rooms, and later replace vacuum tubes with transistors (solid-state devices) which dissipate far less heat. Consume a lot of energy – use more efficient power supplies and cooling, power the machine from a dedicated supply, and again migrate to transistor-based circuitry with lower power consumption. Large in size – replace bulky vacuum tubes and discrete components with transistors and then integrated circuits, use modular/rack packaging and magnetic-core or semiconductor memory to shrink the machine. Slow processing – replace vacuum tubes with faster transistors/ICs, increase clock speed, use parallel processing and faster memory (e.g., core memory instead of delay lines). Limited programming capabilities – provide a stored-program design (von Neumann architecture) with memory holding both data and instructions, use assembly/high-level languages and compilers, and provide input/output devices and an operating system to ease program entry and execution.

Marking points:

• 1 mark (a): Correctly states first generation (vacuum-tube generation).

• 1 mark (b): Names both John Mauchly and J. Presper Eckert (or 'Eckert and Mauchly') as builders; accept University of Pennsylvania/Moore School context.

• 1 mark (c): Identifies vacuum tubes as the main technology.

• 1 mark (d): Gives a valid solution for heat (e.g., cooling/air-conditioning or transistors).

• 1 mark (d): Gives a valid solution for energy consumption (e.g., efficient power supply or transistor/IC replacement).

• 1 mark (d): Gives a valid solution for large size (e.g., transistors/ICs, modular packaging, smaller memory).

• 1 mark (d): Gives a valid solution for slow processing (e.g., faster switching devices, higher clock, parallelism).

• 1 mark (d): Gives a valid solution for limited programming capability (e.g., stored-program concept, high-level

© Hak Cipta Universiti Teknologi MARA

languages/compilers, better I/O).

• 1 mark: Overall coherence/terminology (uses correct computer-organization terms such as vacuum tube, stored program, transistor, IC).

• 1 mark: Completes all subparts (a–d) with no missing problem in the table.

Model confidence: high.

Model note: The question text contains typographical errors in the problem table (e.g., 'Consume a lot of ener', 'Lar e in size', 'Slow rocessin', 'Limited ro rammin ca abilities'); these have been interpreted as energy, large size, slow processing and limited programming capabilities. The statement's 'Mr. X and Mr. Y' is taken to refer to Eckert and Mauchly, the known ENIAC designers. This is an unofficial model answer, not an official UTM marking scheme.

© Hak Cipta Universiti Teknologi MARA

PART B — MARKING GUIDE

QUESTION 11 (20 marks)

Source: CSC429-2025-07, page [5]

Unofficial model answer:

The equation in Figure 1 is read as z = (A² − B*C + jl_)/(D + 5). The OCR text is garbled ('jl_' and 'D+5' appear on a separate line), so the expression is interpreted as a numerator (A² − B*C + jl_) divided by the denominator (D + 5). Because the instruction set in Table 1 contains only ADD, SUB, MUL and DIV (no square or jump instructions), A² must be computed as A*A, and the term jl_ is treated as a single named operand/variable (call it J). Registers A, B, C, D are fixed; x, y, z are temporary, and the final result must end in z. The computation order is: t1 = A*A; t2 = B*C; t3 = t1 − t2; t4 = t3 + J; t5 = D + 5; z = t4 / t5.

(a) THREE-ADDRESS form. Each instruction names two sources and one destination, so no explicit temporaries in memory are needed beyond the named registers:

```
MUL (A, A, x)     ; x = A*A
MUL (B, C, y)     ; y = B*C
SUB (x, y, x)     ; x = A*A - B*C
ADD (x, J, x)     ; x = A*A - B*C + J
ADD (D, 5, y)     ; y = D + 5
DIV (x, y, z)     ; z = (A*A - B*C + J)/(D + 5)
```

This is the most compact form: six instructions, no PUSH/POP traffic.

(b) ONE-ADDRESS form. There is a single implicit accumulator; LOAD/STOR move between accumulator and memory, and ADD p/SUB p/MUL p/DIV p operate accumulator ← accumulator op p. Temporaries x and y hold intermediate values:

```
LOAD A        ; ACC = A
MUL A         ; ACC = A*A
STOR x        ; x = A*A
LOAD B        ; ACC = B
MUL C         ; ACC = B*C
STOR y        ; y = B*C
LOAD x        ; ACC = A*A
SUB y         ; ACC = A*A - B*C
ADD J         ; ACC = A*A - B*C + J
STOR x        ; x = numerator
LOAD D        ; ACC = D
ADD #5        ; ACC = D + 5   (immediate 5; if only memory operands allowed, STOR a constant 5 first)
STOR y        ; y = D + 5
LOAD x        ; ACC = numerator
DIV y         ; ACC = numerator/(D + 5)
STOR z        ; z = final answer
```

(c) ZERO-ADDRESS form. A stack machine: PUSH p pushes operand p, POP p pops top of stack into p, and ADD/SUB/MUL/DIV pop two operands and push the result (second-popped op top-popped). Postfix evaluation of (A*A − B*C + J)/(D + 5):

```
PUSH A
PUSH A
MUL           ; stack: A*A
PUSH B
PUSH C
```

© Hak Cipta Universiti Teknologi MARA

```
MUL         ; stack: A*A, B*C
SUB         ; stack: A*A - B*C
PUSH J
ADD         ; stack: A*A - B*C + J
PUSH D
PUSH 5
ADD         ; stack: numerator, D+5
DIV         ; stack: numerator/(D+5)
POP z       ; z = final answer
```

In all three versions the final result is stored in z, as required, and only the permitted instructions from Table 1 are used.

Marking points:

*• 2 marks: correct interpretation of Figure 1 as z = (A*A - B*C + J)/(D + 5), including handling A^2 as A*A and treating the garbled 'jl_' term as a single operand J (award partial if the denominator grouping is missed).*

• 5 marks: three-address program — correct use of MUL/SUB/ADD/DIV with (p,q,r) operand order, correct evaluation order, temporaries x and y used legally, final DIV writing to z (about 1 mark per correct instruction grouping).

• 6 marks: one-address program — correct LOAD/STOR/ADD p/SUB p/MUL p/DIV p sequence, accumulator used correctly, temporaries x and y for numerator and denominator, final STOR z (deduct for missing STOR or wrong operand order).

• 6 marks: zero-address program — correct postfix ordering with PUSH/POP/ADD/SUB/MUL/DIV, stack discipline correct (two pops, one push per arithmetic op), final POP z.

• 1 mark: overall correctness — all three programs compute the same expression and leave the answer in z, using only instructions listed in Table 1.

Model confidence: medium.

*Model note: Figure 1 is supplied only as a cropped image with garbled OCR text ('z=A2-B*C+jl_' and 'D+5'); the exact numerator term (appearing as 'jl_') and the division grouping are inferred. If the intended expression differs (e.g. the denominator applies only to part of the numerator, or 'jl_' is two operands j and l), the instruction sequences must be adjusted accordingly. The one-address immediate 'ADD #5' assumes an immediate form; if only memory operands are permitted, a constant 5 must be stored to a temporary first. This is an unofficial model answer, not an official UTM marking scheme.*

END OF MARKING SCHEME

© Hak Cipta Universiti Teknologi MARA